

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 แนวคิดของระบบควบคุมภายในบ้าน

2.1.1 แนวคิดและหลักการทั่วไป

ตั้งแต่อดีตถึงปัจจุบันสิ่งที่เราพบเห็นอยู่เสมอก็คือการวิวัฒนาการของเทคโนโลยี สิ่งที่เป็นปัจจัยทำให้เกิดการพัฒนานั้นก็คือความต้องการของมนุษย์เพื่อเพิ่มความสะดวกสบายและความปลอดภัยในชีวิตและทรัพย์สิน ดังจะเห็นได้จากอดีตการซักผ้าเราใช้หิน และไม่ทูปผ้าให้สะอาด ต่อมามีการพัฒนาเครื่องมือเครื่องใช้ให้มีความสามารถมากขึ้น ลดเวลาในการทำงานลง และได้ผลลัพธ์ที่ดีขึ้น เป็นลำดับ

ระบบสมาร์ทโฮม (Smart Home) มาจากแนวคิดจะทำสิ่งต่าง ๆ ภายในบ้าน ให้สามารถควบคุมจัดการได้ง่ายขึ้น จนกระทั่งสามารถทำงานได้อย่างอัตโนมัติ เป็นไปตามความต้องการของผู้อยู่อาศัย และเพิ่มมาตรฐานความเป็นอยู่ในด้านต่าง ๆ ให้ดีขึ้น ทั้งในด้านชีวิต ทรัพย์สิน ความปลอดภัย เป็นต้น

ตัวอย่างเช่น ตอนเช้าตรู่ก่อนที่ผู้อยู่อาศัยจะตื่น แอร์จะถูกลดอุณหภูมิเพื่อประหยัดไฟฟ้า กาต้มน้ำในห้องครัวเริ่มทำงานเพื่อเตรียมพร้อมสำหรับกาแฟอุ่นๆ เมื่อถึงเวลา 6.00 น. ไฟในห้องนอนสว่างขึ้นสลัวๆ บานเกร็ดเปิดเพื่อรับลมยามเช้า ไฟประตูรั้วปิดเพื่อประหยัดพลังงาน สปริงเกอร์เริ่มทำการรดน้ำต้นไม้ แต่ถ้าเมื่อคืนฝนตกซึ่งตรวจจับได้จากเซ็นเซอร์ (Sensor) สปริงเกอร์ก็จะไม่ทำงานเพื่อเป็นการประหยัดน้ำ เป็นต้น

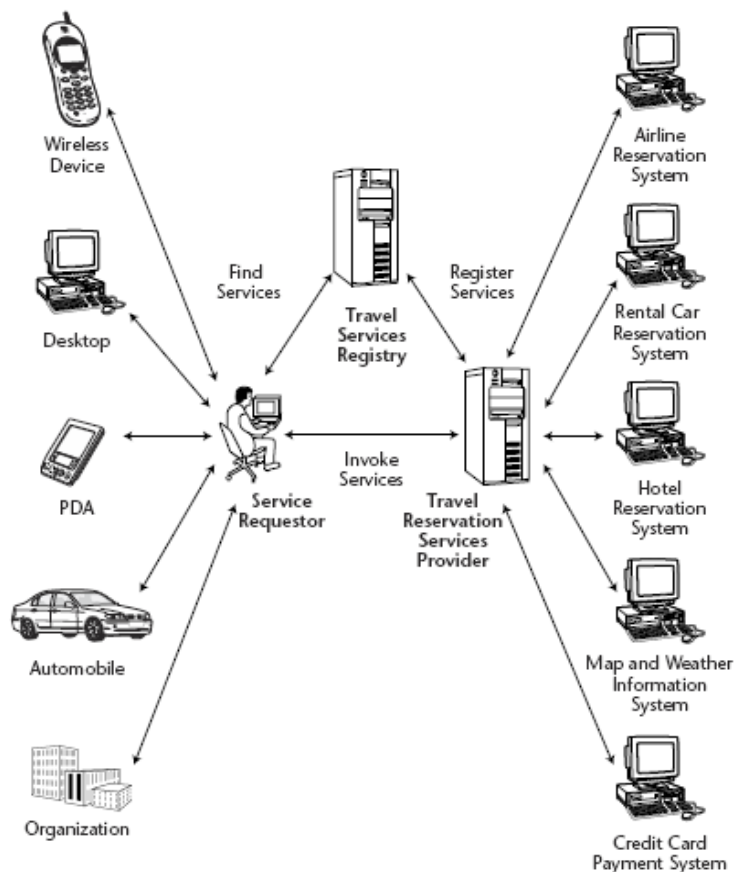
2.1.2 แนวคิดและหลักการของระบบ Smart Home

ระบบรักษาความปลอดภัยและอำนวยความสะดวกผ่านเครือข่าย (Smart Home) จะเน้นที่ระบบควบคุมดูแล (Control and monitor system) ที่รวมเอาความสามารถด้านเครือข่ายและการควบคุมระยะไกลให้สามารถใช้งานได้กับอุปกรณ์ที่หลากหลาย และเอื้ออำนวยในการผสานเข้ากับระบบควบคุมอื่นๆ อันจะนำมาถึงการพัฒนาที่มีความซับซ้อนยิ่งขึ้นในอนาคต ดังเช่น ระบบรักษาความปลอดภัยกลาง (Center Security Control) ที่บริการการดูแลความปลอดภัยให้กับลูกบ้านที่เป็น Smart Home โดยถ้าตรวจความผิดปกติของลูกบ้าน ก็จะสามารถจะควบคุมดูแลอีกชั้นหนึ่ง ซึ่งช่วยให้มีความปลอดภัยมากยิ่งขึ้น เช่น การปิดกั้นทางออกของโจรจากบริเวณ การแจ้งสัญญาณเตือนภัยแก่บริเวณใกล้เคียง เป็นต้น

จากแนวคิดที่กล่าวข้างต้น ในขั้นแรกผู้พัฒนาจะพัฒนาระบบที่เน้นการควบคุมภายในบ้าน แต่ละหลังให้มีความสามารถเพียงพอที่จะทำงานได้เองโดยลำพัง การวางเครือข่ายของการสื่อสาร

ของอุปกรณ์เครื่องใช้ไฟฟ้า การเชื่อมต่อบริการจากภายนอก การให้บริการต่าง ๆ และส่วนในการใช้งานของระบบผ่านอุปกรณ์ที่แพร่หลาย เช่น โทรศัพท์มือถือ เว็บเบราว์เซอร์

2.2 เว็บเซอร์วิส (Web service)



รูปที่ 2.1 แสดงลักษณะการให้บริการของเว็บเซอร์วิส

เว็บเซอร์วิส คือ แอปพลิเคชัน หรือ โปรแกรมที่ทำงานอย่างใดอย่างหนึ่งในลักษณะให้บริการ (Service) ที่จะถูกเรียกใช้งานจากแอปพลิเคชันอื่นๆ ในรูปแบบ RPC (Remote Procedure Call) หรือระบบส่งงานระยะไกล โดยการให้บริการจะมีเอกสาร ที่อธิบายคุณสมบัติของบริการกำกับไว้ มีภาษาที่ถูกใช้เป็นตัวสื่อในการแลกเปลี่ยนคือ XML ทำให้ผู้ใช้สามารถเรียกใช้ส่วนประกอบใดๆ ในแพลตฟอร์มใด ๆ ก็ได้ บนโปรโตคอล HTTP สำหรับ World Wide Web อันเป็นช่องทางที่ได้รับการยอมรับทั่วโลก ในการติดต่อสื่อสารกันระหว่าง แอปพลิเคชัน กับ แอปพลิเคชัน

เว็บเซอร์วิสมีพื้นฐานมาจากแนวคิดของสถาปัตยกรรมการให้บริการหรือเอสโอเอ (SOA-Service-oriented architecture) เป็นแนวคิดใหม่ของการระบบแบบกระจายศูนย์กลาง (Distributed computing) ซึ่งทำให้ซอฟต์แวร์คอมโพเนนต์ (Software components) ฟังก์ชัน, ออบเจกต์ สามารถ

ใช้งานร่วมกันได้จากระบบที่สถาปัตยกรรมมีความแตกต่างกัน โดยใช้ประโยชน์จากอินเทอร์เน็ตเป็นโปรตอคอลเพื่อค้นหา, ลงทะเบียนหรือการเรียกบริการนั้นๆ

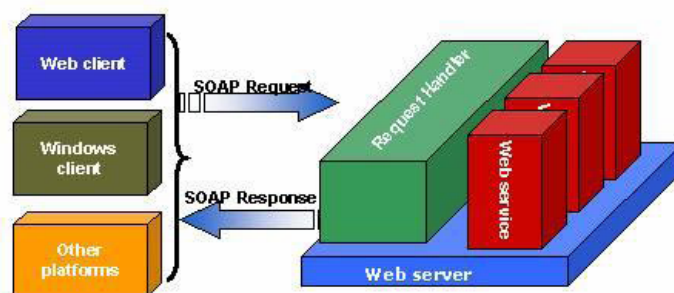
2.2.1 หลักการเว็บเซอร์วิส

เว็บเซอร์วิส (Web Service) คือ การติดต่อระหว่างเครื่องคอมพิวเตอร์ที่กล่าวถึงกลุ่มของ Operation ที่เป็นการเข้าถึงผ่านเครือข่ายโดยอยู่ภายใต้มาตรฐานของ XML messaging หรือเราอาจกล่าวได้

ว่าความหมายโดยรวมแล้ว เว็บเซอร์วิส หมายถึง บริการที่ให้โปรแกรมใช้ติดต่อระหว่างกันผ่านเครือข่าย

WSDL (Web Service Description Language) คือข้อกำหนดที่ใช้สร้างเอกสารสำหรับอธิบายรายละเอียดของ Web Service ซึ่งได้แก่ ชื่อของ Web service , URI, ฟังก์ชันการทำงาน ฯลฯ

UDDI (Universal Description, Discovery, and Integration) คือเป็น Directory สำหรับค้นหาบริการของ web service ทั้งนี้เพื่อให้ web service สามารถถูกค้นหาได้จาก internet เพื่อใช้งาน



รูปที่ 2.2 แสดงโครงสร้างของ Web Service

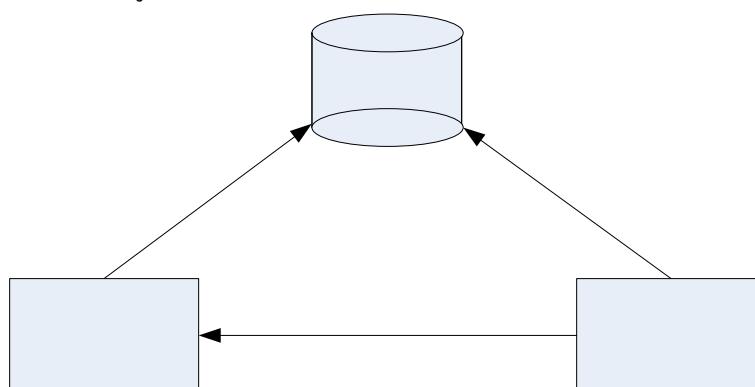
2.2.2 คุณลักษณะของ Web service

- XML Base
- Message Base
- ไม่ขึ้นกับ Programming Language
- สามารถสร้างบริการใหม่ โดยการรวมหลายๆบริการ (aggregation)
- ใช้งานได้ผ่านเครือข่ายอินเทอร์เน็ต
- ใช้มาตรฐานสากล
- รายละเอียดการทำงานของ web service จะถูกซ่อนไว้ โดยผู้ใช้งานจะมองเห็นแค่รูปแบบและลักษณะการติดต่อที่ผู้ให้บริการประกาศไว้เท่านั้น
- ซอฟต์แวร์ที่ทำงานบนระบบ web service สามารถนำมาแก้ไขรายละเอียดภายในได้ โดยไม่ส่งผลกระทบเป็นลูกโซ่ และผู้ใช้ปลายทางก็ไม่จำเป็นต้องโหลดซอฟต์แวร์ติดตัวเกินความจำเป็น

2.2.3 โครงสร้างของเว็บเซอร์วิส

ประกอบไปด้วยส่วนสำคัญ 3 ส่วนคือ ผู้ให้บริการ (Service Provider), ผู้ขอบริการ (Service Requestor), ตัวแทนของผู้ให้บริการ (Service Register) โดยทั้ง 3 ส่วนจะติดต่อกันภายใต้เงื่อนไขดังนี้

1. ผู้ให้บริการทำการประกาศบริการไปยังตัวแทนของผู้ให้บริการ ซึ่งจะถูกบันทึกไว้ใน Service Directory ของตัวแทนผู้ให้บริการ
2. ผู้ขอใช้บริการจะทำการค้นหาบริการจากตัวแทนของผู้ให้บริการ
3. เมื่อพบบริการที่ต้องการแล้ว ผู้ขอใช้บริการจะทำการติดต่อไปยังผู้ให้บริการ โดยร้องขอบริการไปยังผู้ให้บริการนั้น



รูปที่ 2.3 แสดงลักษณะการติดต่อใช้บริการ Web Service

2.3 XML

สำหรับประโยชน์ของ XML นั้น เป็นด้านความยืดหยุ่นในการใช้งานสำหรับแอปพลิเคชัน ที่อิงกับ Web Base ที่ใช้ง่ายในการค้นหาข้อมูล มีความยืดหยุ่นในการพัฒนาเว็บ สามารถผสมผสานข้อมูลจากหลายแหล่ง จากแอปพลิเคชันที่ต่างกัน สามารถแสดงข้อมูลแบบต่างๆ และสามารถ update ข้อมูลให้ทันสมัยเสมอ และคาดว่าจะเป็มาตรฐานใหม่ของระบบเปิด ซึ่งนับเป็น format ใหม่สำหรับการส่งข้อมูลบนเว็บที่มากด้วยข้อมูลหลายแบบ แต่ส่งผ่านด้วยเทคโนโลยีที่บีบอัดข้อมูลที่ให้ความเร็วได้รับการสนับสนุนจากผลิตภัณฑ์ค่ายไมโครซอฟท์

XML เป็นการทำงานในระดับกลาง middle tier ที่สามารถเรียกใช้งานข้อมูลได้หลากหลาย ระบบฐานข้อมูลและโอนข้อมูลให้อยู่ในรูปแบบของ XML และมีการให้รายละเอียดเกี่ยวกับตัว ข้อมูลโครงสร้างต่างๆ ของระบบฐานข้อมูลได้ XML เป็นระบบเปิดที่นำเสนอข้อมูลในรูปแบบ text ผ่านทาง HTTP เหมือนกับ HTML แต่จะมีคุณสมบัติในการให้ข้อมูลแบบ real time อัปเดตหรือเปลี่ยนแปลงได้ตามข้อ

Web Service

3. Client

กำหนด การแสดงข้อมูลจาก XML ใน HTML จะเป็นการเพิ่มในส่วนของรายละเอียดข้อมูล ที่มีการเรียก

ใช้จากแหล่งหรือฐานข้อมูลที่เชื่อมโยงกันในหลายแหล่งเพื่อให้ HTML มีความสมบูรณ์มากขึ้น

ในโอกาสการพัฒนาเว็บหรือการเขียน และสร้าง HTML ไม่จำเป็นต้องมีการเขียนชุดคำสั่งที่ยุ่งยากซับซ้อนมากก็สามารถทำงานร่วมกับระบบข้อมูลได้อย่างมีประสิทธิภาพ XML จะทำการกำหนดค่าสำหรับโครงสร้าง ข้อมูลที่จะนำไปแสดงใน HTML นอกจากนั้นยังสามารถนำไปสนับสนุนระบบการแลกเปลี่ยนข้อมูลข่าวสารทาง Electronic ได้อย่างดีอีกด้วย

2.3.1 แนะนำ XML

XML ย่อมาจาก eXtensible Markup Language XML ถูกออกแบบมาเพื่อใช้อธิบายข้อมูล ใน XML จะไม่มีแท็ก (tag) ที่กำหนดไว้ล่วงหน้า เราจะต้องสร้างแท็กของเราขึ้นมาเอง และจะใช้ Document Type Definition (DTD) ในการอธิบายรูปแบบของข้อมูล

ข้อแตกต่างที่สำคัญระหว่าง XML และ HTML คือ XML ถูกออกแบบมาเพื่อใช้แลกเปลี่ยนข้อมูล

ไม่ใช่ว่าแทนที่ HTML XML กับ HTML ถูกออกแบบมาเพื่อจุดประสงค์ที่ต่างกัน HTML จะเกี่ยวข้องกับการแสดงข้อมูล XML จะเกี่ยวข้องกับการอธิบายข้อมูล แท็กของ HTML เช่น `<H1></H1>` เป็นแท็กสำหรับบอกรูปแบบการแสดงผล ในขณะที่แท็กของ XML ต้องสร้างขึ้นมาเอง เช่น

```
<BOOK>
  <NAME>Beginning XML</NAME>
  <ISDN>1-861003-41-2</ISDN>
  <PAGE>800</PAGE>
</BOOK>
```

จะใช้แสดงข้อมูลของหนังสือที่ประกอบด้วย element BOOK , NAME , ISDN และ PAGE ข้อมูลที่อยู่ในรูปแท็กนี้เป็นอิสระต่อซอฟต์แวร์และฮาร์ดแวร์ จึงเหมาะที่จะนำมาใช้ในการแลกเปลี่ยนข้อมูลระหว่างระบบบนอินเทอร์เน็ต การเปลี่ยนข้อมูลให้อยู่ในรูป XML เป็นการลดความซับซ้อนและสามารถสร้างข้อมูลที่ถูกรับโดยแอปพลิเคชันใด ๆ ก็ได้ นอกจากนั้นการเพิ่มเติม element จะไม่ทำให้แอปพลิเคชันเสียหาย เนื่องจากการอ่านข้อมูลโดย XML parser จะใช้วิธีแยกข้อมูลจาก element ที่ต้องการ จาก XML ข้างต้นนี้จะแยกข้อมูล element NAME , ISDN และ PAGE หากเพิ่ม element AUTHOR เข้าไป

แอปพลิเคชันก็ยังคงดึงข้อมูลใน element เดิมโดยจะมองข้าม element ที่ไม่ได้กำหนดให้แยกข้อมูลออกมาในตอนพัฒนาแอปพลิเคชัน

2.3.2 โครงสร้างของ XML

เอกสาร XML จะถูกสร้างจากส่วนต่างๆ ดังนี้

- Element เป็นโครงสร้างหลักของ XML
- Tag ใช้ในการกำหนด markup element
- Attribute ให้ข้อมูลเพิ่มเติมกับ element
- Entity เป็นตัวแปรที่ใช้ในการกำหนดเท็กซ์ทั่ว ๆ ไป entity reference เป็นการอ้างถึง entity เช่น entity ชื่อ domain มีค่าเท่ากับ www.w3.org entity reference คือ &domain หากภายในเอกสาร XML มี &domain ค่าของมันจะมีค่าเท่ากับ www.w3.org
- PCDATA -หมายถึง parse character data PCDATA เป็นเท็กซ์ที่จะถูก parse โดย parser
- CDATA -หมายถึง character data เป็นเท็กซ์ที่จะไม่ถูก parse โดย

2.4 ASP.NET

เว็บเซอร์วิสช่วยให้เราเรียกใช้ ซอฟต์แวร์คอมพิวเตอร์ ผ่านทางเว็บ โปรโตคอลเช่น HTTP โดยใช้อินเทอร์เน็ตและ XML เราสามารถสร้าง software component เพื่อติดต่อกับคนอื่นๆ โดยไม่ต้องสนใจในเรื่องของภาษา และแพลตฟอร์ม จากเดิมแต่ละค่ายจะมีการทำการติดต่อผ่านโปรโตคอลของค่ายตัวเอง มีอินเทอร์เน็ตที่แตกต่างกันไป ในเว็บเซอร์วิสสิ่งเหล่านี้จะเป็นมาตรฐานในการติดต่อส่งผ่านกันให้เป็นเรื่องง่าย โดยผ่าน SOAP และ XML เป็นเครื่องมือสำคัญ

2.4.1 องค์ประกอบในการใช้งาน .NET เว็บเซอร์วิส

รูปแบบการเชื่อมต่อ

Microsoft .NET Web Services ตอนนี้อยู่รับสามโปรโตคอล ได้แก่ HTTP GET, HTTP POST และ SOAP (Simple Object Access Protocol) เนื่องจากโปรโตคอลเหล่านี้เป็นมาตรฐานของเว็บอยู่แล้ว จึงง่ายที่ไคลเอนต์จะเรียกใช้

HTTP GET

query string จะรวมอยู่ใน URL ของเซิร์ฟเวอร์นั้น

HTTP POST

จะทำการรวม ชื่อ ค่าตัวแปรต่างๆ ไว้ในส่วนของบอดี ของ HTTP request message

SOAP

SOAP จะแตกต่างจากทั้งสองวิธีข้างต้น มันจะใช้ XML ในการจัดวางรูปแบบเมสเสจที่ส่งไปจะมีโครงสร้างที่ดีกว่าและสามารถส่งผ่านข้อมูลที่ซับซ้อนได้ ข้อแตกต่างอีกอย่างคือ SOAP สามารถใช้บน transport protocols อื่นๆ ได้ เช่น SMTP นอกเหนือจาก HTTP

2.4.2 การสร้าง ASP.NET เว็บเซอร์วิส

ขั้นตอนในการสร้าง Web Service

1. สร้างไฟล์ .asmx สำหรับ web service ที่สร้างด้วย ASP.NET ต้องประกอบไปด้วย directive `<% webService . . . %>` การเขียนก็คล้ายกับการเขียนคลาส
2. WebService Class นั้นต้องทำการ inherit มาจาก System.Web.Services namespace รวมทั้งทำการกำหนด namespace ของเว็บเซอร์วิสของคุณด้วยเนื่องจาก default ของมันคือ `http://tempuri.org/` ซึ่งจะไม่ unique
3. เมธอดที่ต้องการให้เป็น เมธอดสาธารณะที่คนอื่น ๆ เข้ามาเรียกใช้ได้ ต้องทำการกำหนดให้เป็น `<Web Method>` ด้วย
4. นำไฟล์ .asmx ที่สร้างไปไว้ในไคลเอนท์ที่ต้องการใน IIS

ตัวอย่างเว็บเซอร์วิส

```
<%@ WebService Language="VB" Class="Math"%>
Imports System.Web.Services
Imports System
<WebService(Namespace:="http://www.olala.com/")> _
Public Class Math
<WebMethod()> _
Public Function Add(num1 As Integer, num2 As Integer) _
As Integer
Return num1 + num2
End Function
<WebMethod()> _
Public Function Hello() _
As String
Return "Hello World"
End Function
End Class
```

การกำหนดค่าแอพริพริวต์ต่างๆให้แก่เว็บเซอร์วิส

- BufferResponse (boolean) ควบคุมให้มี buffer ของผลตอบรับของเมธอดหรือไม่
- CacheDuration กำหนดค่าช่วงเวลา(วินาที) ที่ทำการเก็บ method response ใน cache ค่าปกติคือ 0 วินาที
- Description กำหนดคำอธิบายของเมธอดนี้
- EnableSession (boolean) ให้มีการใช้ เซสชันสำหรับเมธอดนี้หรือไม่ โดยค่าปกติเป็น true ถ้าเราไม่ได้ ใช้งานควรกำหนดเป็น false เพื่อเพิ่มประสิทธิภาพ
- MessageName เพื่อใช้ในการแย่งแยกเว็บเมธอดในกรณีที่มี ชื่อของเมธอดเหมือนกัน
- TransactionOption เป็นการกำหนด transaction ของเมธอดนั้น มีทั้งหมด 5 แบบ ดังที่ได้กล่าวไปแล้วในเรื่อง Transaction ได้แก่

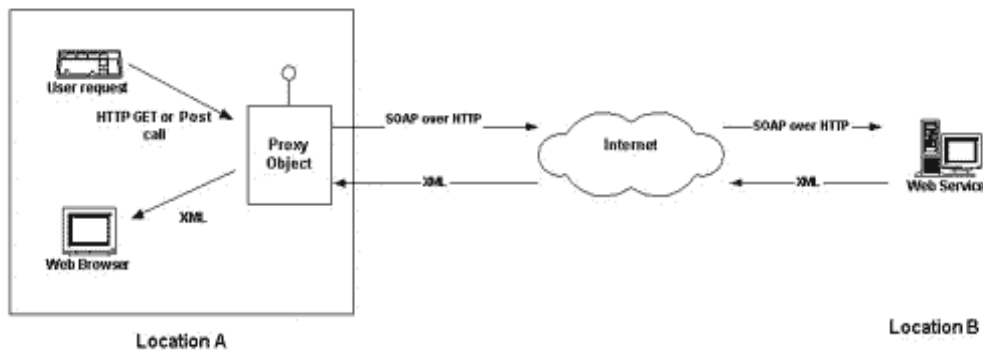
1. Disabled object ที่สร้างขึ้นจะไม่มี transaction ใน COM+ transaction แต่สามารถติดต่อโดยตรงกับ DTC ในกรณีที่ต้องการการสนับสนุน
2. NotSupported ไม่มีการทำ transaction เลย
3. Supported ถ้า object นั้นจะถูกรันใน context transaction ของตัวที่สร้างมันขึ้นซึ่งมาจากการทำ transaction แต่ถ้าตัวมันเอง เป็นตัวเริ่ม (root ไม่ได้ถูกใครสร้างมาอีกที) หรือ context ที่สร้างมัน ไม่ทำ transaction มันก็จะไม่ทำ transaction
4. Required ถ้า object นั้นจะถูกรันใน context transaction ของตัวที่สร้างมันขึ้นซึ่งมาจากการทำ transaction , แต่ถ้าตัวมันเอง เป็นตัวเริ่ม (root ไม่ได้ถูกใครสร้างมาอีกที) หรือ context ที่สร้างมัน ไม่ทำ transaction มันก็จะทำ transaction ใหม่
5. RequiresNew - บอกว่า object ต้องทำ transaction ใหม่

2.4.3 การเรียกใช้งานเว็บเซอร์วิส

ในการเรียกใช้งานเว็บเซอร์วิสนั้นเราสามารถที่จะเรียกใช้ได้ทั้ง 3 แบบ คือ HTTP GET, HTTP POST , SOAP

HTTP GET นั้นเราสามารถเรียกใช้ Web Service ได้โดยตรงผ่านทาง URL โดยใช้ parameter

รวมไปด้วยเลย เช่น <http://localhost/Xtest.asmx/Add?num1=3&num2=6> แต่ในส่วนของ HTTP POST และ SOAP นั้นเราต้องทำการสร้าง Proxy Class ขึ้นมาก่อนเพื่อทำการติดต่อกับ Web Service



รูปที่ 2.4 แสดงการติดต่อจากไคลเอนต์ไปยังเว็บเซิร์ฟเวอร์ผ่านพร็อกซีอ็อบเจกต์

ขั้นตอนการสร้าง Proxy Class

1. ใช้ wsdl.exe ในการคอมไพล์โดยใช้พารามิเตอร์เป็น ชนิดโปรโตคอล, ตำแหน่งของเอกสาร wsdl ภาษาที่ใช้ และอื่นๆ
2. จะได้ไฟล์ออกมา .cs ซึ่งเป็นไฟล์ของ C# เราใช้ตัวคอมไพล์ของ C# ก็จะได้ไฟล์ .dll ออกมา
3. นำไฟล์ .dll นี้ไป add reference ในโปรเจกต์ที่จะเรียกใช้

2.4.4 การจัดการสเตทในเว็บเซอร์วิส

ในส่วนของ ASP.NET มีฟีเจอร์ในการจัดการสเตท ทั้งในส่วนในระดับแอปพลิเคชันและระดับของเซสชันของผู้ใช้ ในส่วนนี้เราจะกล่าวถึงการใช้งานสเตทของ ASP.NET เพื่อนำมาพัฒนาสเตทฟูลเว็บเซอร์วิส

คำว่า สเตท นั้น เราอาจจะนิยามได้ว่า มันคือความสามารถของอ็อบเจกต์, ตัวแปร หรือคอนเทน

เนอร์อื่นๆที่สามารถจดจำค่าของมันได้ ซึ่งเรามีอยู่สองระดับคือ แอปพลิเคชัน และเซสชัน ในระดับแอป

พลิเคชันนั้น เปรียบเหมือนร้านอาหาร บริการต้องจำได้ว่า วันนี้ร้านมีเมนูพิเศษอะไรบ้าง ซึ่งจะนำไปบอก

ลูกค้าทุกโต๊ะเหมือนกันหมด แต่ระดับเซสชัน บริการจะจำได้ว่า ลูกค้าคนนี้สั่งอะไร แต่ละคนสั่งคนละ

อย่างไม่เกี่ยวกันทั้งสองระดับที่กล่าวมานั้นใน ASP.NET เรียกว่า แอปพลิเคชันอ็อบเจกต์ ซึ่ง

ให้บริการ กระบวนการในกาจัดเก็บข้อมูล ซึ่งสามารถเข้าถึงได้จากทุกโค้ด ที่อยู่ในเว็บแอปพลิเคชันนั้น และ เซสชันอ็อบเจกต์ ซึ่งจัดเก็บข้อมูลตามเซสชันของไคลเอนต์แต่ละคนที่เข้ามา

2.4.5 แอปพลิเคชันอ็อบเจกต์

แอปพลิเคชัน อ็อบเจกต์จะถูกสร้างขึ้น หลังจากเกิดการเรียกใช้เว็บหรือเว็บเซอร์วิสขึ้น เป็นครั้งแรกจนกระทั่ง ปิดระบบเว็บเซอร์วิสลง ไป ซึ่งแอปพลิเคชันอ็อบเจกต์นั้นให้การข้อดีต่างๆ ดังเช่น

- ง่ายต่อการเรียกใช้สแตท และสามารถได้จากภาษาใดๆก็ได้ที่ .NET รองรับ
- มีฟังก์ชันในการจัดการ ซึ่งโครโนเซชั่น ทำให้นักพัฒนาง่ายต่อการจัดการการเข้าถึง ตัวแปรแบบโอบอลใน แอปพลิเคชันสแตท
- แอปพลิเคชันสแตท เข้าถึงได้เพียงโค้ดในส่วน of แอปพลิเคชันนั้นๆเท่านั้น ไม่สามารถที่แอปพลิเคชันอื่นจะเข้าถึงได้

ตัวอย่างการเรียกใช้งานแอปพลิเคชัน อ็อบเจกต์

```
Application["DatabaseName"] = "DataStore" ;
```

2.4.6 เซสชันอ็อบเจกต์

เซสชันหมายถึง ช่วงเวลาที่ผู้ใช้คนหนึ่งๆทำการติดต่อกับเว็บแอปพลิเคชันหนึ่ง ซึ่งใน ASP.NET มีการทำงานที่ดีขึ้นกว่า ASP เวอร์ชันก่อนหน้าดังนี้

- เซสชัน อ็อบเจกต์โปรเซสจะถูกแยกออกมา ไม่ได้u3629 อยู่ในโปรเซสของ ASP เหมือนของเดิม ดังนั้นถ้าโปรเซสของ ASP ล้มเหลว ตัวเซสชันก็ยังอยู่
- เซสชัน อ็อบเจกต์ใน ASP.NET ให้การสนับสนุนการใช้ เซิร์ฟเวอร์หลายๆตัว(server farm) จากแนวคิดของ out-of-process ข้างต้น เราจึงสามารถทำให้ เซิร์ฟเวอร์หลายๆตัวใช้ เซสชันสแตท ร่วมกันได้
- เซสชันอ็อบเจกต์ สามารถเรียกใช้โดยไม่ผ่าน คูกี้ ได้ง่ายการจัดการของเซสชันนั้น จะบอกได้จาก เลขเซสชันไอดี ซึ่งเป็นเลข 128 บิต หรือเรารู้จักกันดีว่า GUID's(Globally Unique Identifier) ซึ่งถูกสร้างขึ้นจากอัลกอริทึมที่มั่นใจว่าจะไม่เกิดการซ้ำกัน เลขเซสชันนี้จะถูกจัดเก็บในสองรูปแบบ คือ

1. ผ่านคูกี้ ซึ่งคูกี้ นั้นสามารถบ่งบอกถึงเซสชันของไคลเอนต์
2. จากการรวมค่าของไอดี เข้าไปยัง URL โดยตรง

เว็บเซอร์วิสนั้นก็ต้องการตัวในการจัดการสแตทกับไคลเอนต์ เครื่องไคลเอนต์ที่จะเรียกใช้เว็บ

เซอร์วิสต้องสนับสนุนการใช้งานคูกี้ด้วย เพื่อที่จะเก็บ เซสชันไอดี

ตัวอย่างการใช้งาน เซสชันอ็อบเจกต์ใน ASP.NET

```
Session["UserName"] = "John Smith" ;
```

2.4.7 โหมดการจัดการเซสชันและการคอนฟิกค่า

การคอนฟิกค่าต่างๆใน ASP.NET แบ่งออกได้เป็นสองส่วนด้วยกัน คือ machine.config และ web.config ไฟล์แรกนั้นจะส่งผลกับทุกเว็บแอปพลิเคชันในเครื่อง และ ไฟล์หลังจะเป็นการคอนฟิกเฉพาะ

แอปพลิเคชันซึ่งจะทับแมชชีนคอนฟิกอีกที

โหมดในการใช้งานเซสชันนั้นมีด้วยกัน 3 โหมด ได้แก่

1. In Proc เป็นค่าเริ่มต้นที่ติดตั้ง ก็จะจัดเก็บเซสชันในโปรเซสเดียวกับเว็บเซิร์ฟเวอร์
2. State Server ทำการจัดเก็บลงในแบบ out-of-process ลงในเมมโมรี่ โดยเครื่องที่จะทำเซิร์ฟเวอร์นี้ สามารถเป็นเครื่องเดียวกับ เว็บเซิร์ฟเวอร์ได้

stateConnectionString="tcpip=127.0.0.1:42424"

3. SqlServer ทำการจัดเก็บลงในแบบ out-of-process เช่นกัน แต่จัดเก็บลงในดาต้าเบส

sqlConnectionString="data source=127.0.0.1;user id=sa;password="

ค่าอื่นๆที่สามารถปรับแต่งได้ก็คือ โหมดของการใช้ cookie ว่าจะเป็นแบบ ใช้cookie หรือว่าผ่าน url และการกำหนดค่าช่วงเวลาของcookie

cookieless="true"

timeout="40"

ในเรื่องของประสิทธิภาพ โหมด In Proc มีประสิทธิภาพมากกว่า State Server และ SqlServer ตามลำดับ แต่ สองโหมดหลังจะสามารถใช้กับแบบ เซิร์ฟเวอร์ฟาร์มได้ โหมด SqlServer ก็จะช้าที่สุดแต่ก็มีความน่าเชื่อถือที่สุดเช่นกัน

2.5 ไมโครคอนโทรลเลอร์

ไมโครคอนโทรลเลอร์ตระกูล MCS -51 เป็น ไมโครคอนโทรลเลอร์ แบบชิพเดียวที่ผลิตโดยบริษัทอินเทลมีอยู่ด้วยกันหลายเบอร์ด้วยกันซึ่งแต่ละเบอร์นั้นมีคุณสมบัติที่แตกต่างกันตามแต่จะเลือกใช้งานให้เหมาะสมโดยทั่วไปแล้วคุณสมบัติที่แตกต่างกันในแต่ละเบอร์ก็คือ จำนวนหน่วยความจำรวมและแรม, บิต อินพุตเอาต์พุต

2.5.1 คุณสมบัติของไมโครคอนโทรลเลอร์ MCS-51

- ต้องการแหล่งจ่ายไฟ +5 โวลต์ ชุดเดียว
- มีหน่วยความจำโปรแกรม (Program Memory) ขนาด 4 กิโลไบต์สำหรับเบอร์ 8051 และ 8031

สำหรับเบอร์ 8052 มีหน่วยความจำถึง 8 กิโลไบต์-

- มีหน่วยความจำสำหรับเก็บข้อมูล (Data Memory) ขนาด 128 ไบต์ สำหรับเบอร์ 8052 ขึ้นไปมีถึง 256 ไบต์
- หน่วยความจำสำหรับเก็บโปรแกรมและข้อมูลแยกจากกันอย่างละ 64 กิโลไบต์
- มีไทม์เมอร์ เคนต์เตอร์ ขนาด 16 บิต 2 ชุด (สำหรับ 8052 มี 3 ชุด) ทำงานได้ 4 โหมด
- รับอินเตอร์รัพท์ได้ 6 แหล่ง 5 เวกเตอร์ สำหรับเบอร์ 8052 ขึ้นไปมี 8 แหล่ง 6 เวกเตอร์
- มีพอร์ตรับส่งข้อมูลอนุกรม (UART) 2 พอร์ตแบบรับ-ส่งพร้อมกัน 2 ทิศทาง (Full Duplex) เลือกรูปแบบได้ 4 โหมด
- มีคำสั่งในการทำ AND, OR หรือ Complement ได้ทั้งแบบ 8 บิตและ 1 บิต

2.5.2 โครงสร้างของไมโครคอนโทรลเลอร์ตระกูล MCS-51

เราจะกล่าวถึงรายละเอียดคร่าวๆของไมโครคอนโทรลเลอร์ตระกูล MCS - 51 โดยมีจุดประสงค์เพื่อให้ผู้อ่านทำความเข้าใจและเห็นภาพกว้างๆ ของไมโครคอนโทรลเลอร์ตระกูลนี้เพื่อเป็นพื้นฐานในการศึกษารายละเอียดต่อไป ไมโครคอนโทรลเลอร์ตระกูล MCS -51 มีสมาชิกในตระกูลหลายเบอร์ด้วยกันแต่ละเบอร์จะมีคุณสมบัติพิเศษบางอย่างแตกต่างกัน เช่น มีหน่วยความจำภายในสำหรับเก็บโปรแกรมและข้อมูลภายในชิพเพิ่มขึ้น มีวงจรเปลี่ยนค่าสัญญาณอนาล็อกเป็นดิจิทัลในตัว สามารถรับสัญญาณอินเตอร์รัพท์ได้หลายชนิด ทำกระบวนการ DMA (Direct Memory Access) ได้ในตัว มีรีจิสเตอร์สำหรับใช้เป็นไทม์เมอร์หรือเคนต์เตอร์เพิ่มขึ้น คุณสมบัติพิเศษที่แตกต่างกันของไมโครคอนโทรลเลอร์แต่ละเบอร์ในตระกูล นี้

ไมโครคอนโทรลเลอร์เบอร์ที่นับได้ว่าเป็นเบอร์พื้นฐานสำหรับตระกูล MCS -51 นี้ได้แก่ เบอร์ 8051, 8031 , 8751 โดยเบอร์ 8051 จัดเป็นสมาชิกตัวแรกในตระกูลซึ่งมีหน่วยความจำสำหรับเก็บโปรแกรมภายในชิพเป็น ROM ขนาด 4 กิโลไบต์ และหน่วยความจำ สำหรับเก็บข้อมูลทั่วไปภายใน MCS -51 (RAM) เองจำนวน 128 ไบต์ มีพอร์ตขนาด 8 บิต 4 พอร์ต มีรีจิสเตอร์สำหรับใช้เป็นไทม์เมอร์หรือเคนต์เตอร์ขนาด 16 บิตรวม 2 ตัว รับสัญญาณอินเตอร์รัพท์จากภายนอกได้ 2 ชนิดสามารถรับส่งข้อมูลแบบอนุกรมผ่านทางพอร์ตสื่อสารข้อมูลแบบอนุกรม มีวงจรออสซิลเลเตอร์เพื่อ สร้างสัญญาณนาฬิกาควบคุมการทำงานในตัว ส่วนเบอร์ 8751 จะมีคุณสมบัติเหมือนเบอร์ 8051 ทุกอย่างต่างกันเพียงชนิดของหน่วยความจำสำหรับเก็บโปรแกรมภายในชิพของเบอร์ 8751 จะเป็นอีพროมแทนที่จะเป็นรอมส่วนเบอร์ 8031 จะเหมือนกับเบอร์ 8051 ต่างเพียงในเบอร์ 8031 ไม่มีหน่วยความจำสำหรับเก็บโปรแกรมภายในชิพเท่านั้น

ไมโครคอนโทรลเลอร์ตระกูล MCS -51 ทุกเบอร์ใช้แรงดันไฟเพียง 5 โวลต์ในการทำงาน ส่วน

กระแสไฟฟ้าที่ใช้จะแตกต่างกันไปตามชนิดของเทคโนโลยีที่ใช้ในการผลิต เบอร์ของไมโครคอนโทรลเลอร์ตระกูลนี้ที่มีตัวอักษร C อยู่ตรงกลางเบอร์ เช่น 80C31, 80C51 จะเป็นเบอร์ของชิพที่

ผลิตโดยอัสยเทคโนโลยี CHMOS ซึ่งใช้พลังงานในการทำงานน้อยกว่าและสามารถควบคุมการใช้พลังงานของตัวชิพได้จากโปรแกรมเพื่อการประหยัดพลังงานในระบบ

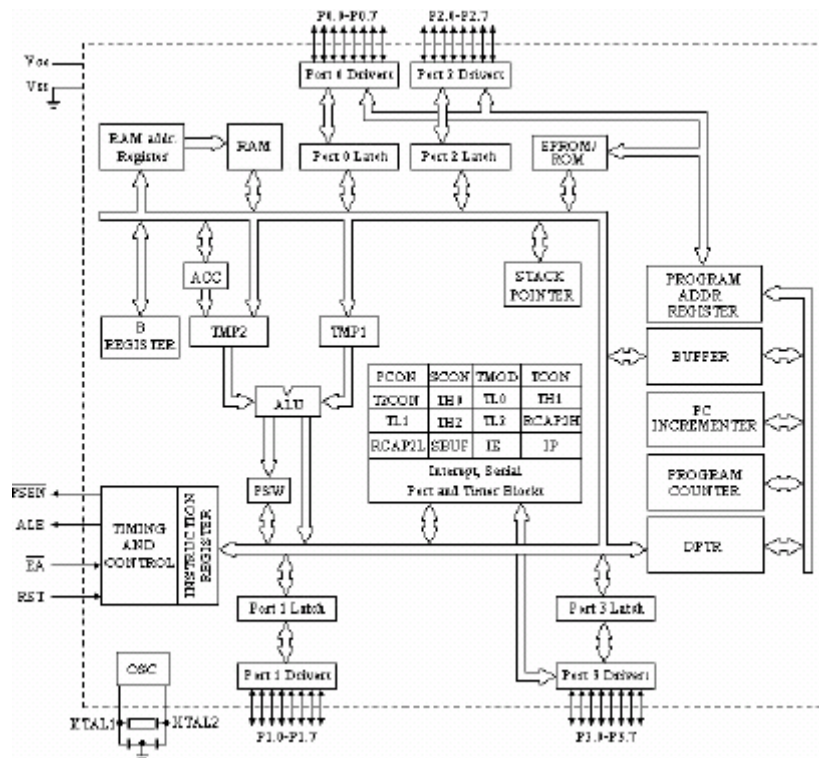
MCS -51 เป็นตระกูลของไมโครคอนโทรลเลอร์ที่ถูกพัฒนาขึ้นมาจากตระกูล MCS -48 ดังนั้นจึงมีความสามารถเหนือกว่าหลายอย่าง เช่น ความเร็วในการประมวลผลของ MCS -51 สามารถใช้ความถี่ได้ถึง 12เมกะเฮิร์ตซ์ หรือบางเบอร์ในตระกูล สามารถใช้ได้ถึง 16 เมกะเฮิร์ตซ์ ทำให้ช่วงระยะเวลาในการทำงานแต่ละคำสั่งน้อยมาก

หน้าที่การใช้งานแต่ละขาของชิพไมโครคอนโทรลเลอร์ตระกูล MCS -51 มีดังนี้

- ขา Vss (ขา 20) สำหรับ ต่อลง GND
- ขา Vcc (ขา 40) สำหรับ ต่อลงแหล่งจ่ายกระแสตรงขนาด 5V (DC)
- ขา พอร์ต 0 (ขา 32 - 39) มี 8 ขา ใช้เป็นขาสำหรับพอร์ต 0 บิตขนาด 8 บิต (P0.0 – P0.7) แบบ Open Drain Bidirectional พอร์ตนี้สามารถใช้งานเป็นอินพุตและเอาต์พุตพอร์ตทั่วไปได้โดยหากใช้งานเป็นอินพุตพอร์ต ต้องโหลดค่า 1 ไปยังแต่ละบิตของพอร์ตนี้เพื่อบังคับให้ขาอยู่ในสถานะถูกปล่อยลอย (มีสถานะ High Impedance) นอกจากใช้งานเป็น อินพุตและเอาต์พุตพอร์ตแล้ว พอร์ต 0 ยังใช้ในการติดต่อหน่วยความจำสำหรับเก็บโปรแกรมและข้อมูลภายนอกชิพด้วย โดยส่งค่าแอดเดรสไบต์ค่า (A0 – A7) และมีลิตเฟลกซ์กับการรับข้อมูล (D0 – D7) จากหน่วยความจำภายนอกในระหว่างการเขียนหรืออ่านข้อมูลโดยมีวงจรพุลอัฟภายใน
- ขาพอร์ต 1 (ขา 1 – 8) มี 8 ขา ใช้เป็นขาสำหรับพอร์ต 1 (P1.0 – P1.7) สามารถใช้งานเป็นอินพุตและเอาต์พุตพอร์ต

2.5.3 โครงสร้างภายในของ 8051

ไมโครคอนโทรลเลอร์ MCS-51 ใช้เทคโนโลยีในการผลิตเป็นแบบ NMOS และ CMOS เบอร์ 8032 และ 8052 จะมีรอมแบบพื้นฐาน อยู่ภายในจึงสะดวกสำหรับโปรแกรมเมอร์ที่จะเขียนโปรแกรมด้วยภาษาเบสิก โครงสร้างภายในสำหรับเบอร์ 8051 เป็นดังรูปที่ 2-10 โครงสร้างภายในของ 8051

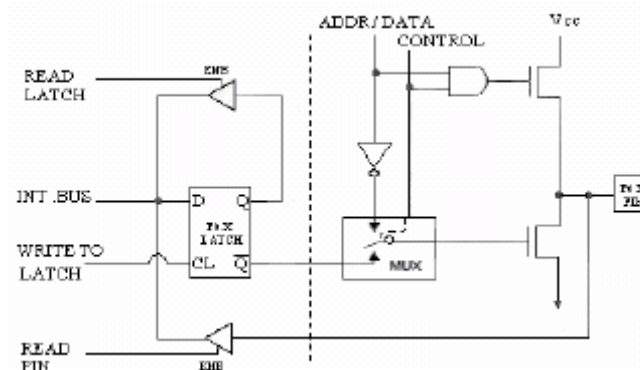


รูปที่ 2.5 แสดงบล็อกไดอะแกรมของ MCS 8051

Port ต่างๆ ภายใน 8051

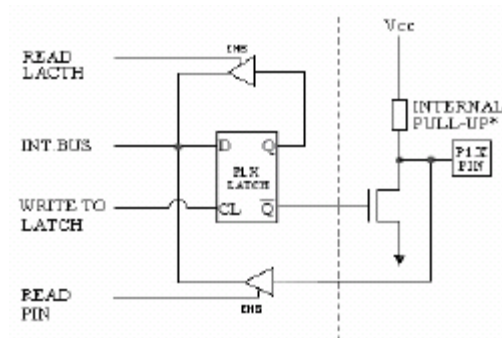
พอร์ทของ 8051 เป็น ไมโครคอนโทรลเลอร์ ขนาด 40 ขา ซึ่งมีขาต่าง ๆ ดังนี้

- Vcc (ขา 40) ต่อกับ +5V
- Vss (ขา 20) เป็นขา GND
- พอร์ท 0 (ขา 32-39) มีทั้งหมด 8 บิต คือ P0.7- P0.0 ใช้งานได้ 2 หน้าที่คือ แอดเดรส บัสและดาต้าบัส เมื่อต้องการให้ทำงานเป็นอินพุตพอร์ทต้องส่งลอจิก “1” ไปยังพอร์ท นี้จะมีผลให้ Q bar ของ D-FF เป็น “0” ทำให้ FET ตัวล่างมีสถานะเป็น OFF สัญญาณ ที่ใช้อ่านอินพุตพอร์ทแลตช์ โดยส่งสัญญาณ READ LATCH ไปกระตุ้นที่ Tri State Buffer ตัวบนและการอ่านพอร์ท(pin) จะใช้สัญญาณ Read(pin)



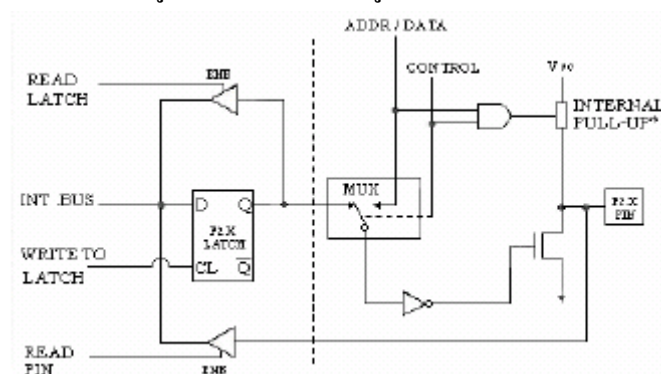
รูปที่ 2.6 แสดงโครงสร้าง port 0 (bit)

- พอร์ต 1 (ขา 1-8) มีทั้งหมด 8 บิตคือ (P1.0- P1.7) มีโครงสร้างคล้าย พอร์ต 0 แต่จะใช้ความต้านทานภายในพูลอัพแทน (Internal Pull up Register) มีโครงสร้างดังรูปที่ 2-12



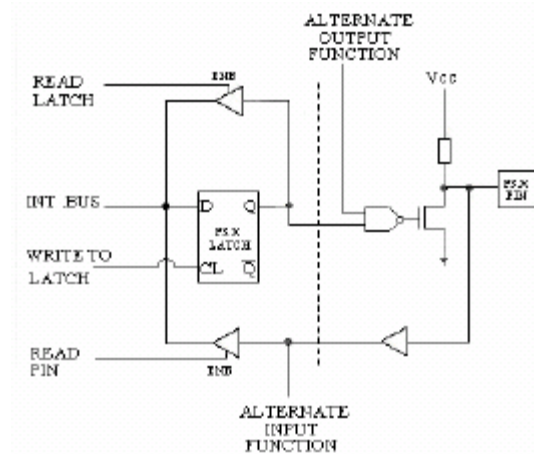
รูปที่ 2.7 แสดงโครงสร้าง port 1 (bit)

- พอร์ต 2 (ขา 21-28) มีทั้งหมด 8 บิต คือขา (P2.7- P2.0) มีโครงสร้างคล้าย พอร์ต 0 โดยมี FET ตัวกลางตัวเดียวส่วนด้านบนใช้ความต้านทานพูลอัพแทน (Internal Pull up) พอร์ตนี้ทำงาน 2 หน้าที่ คือ สามารถใช้เป็น แอดเดรสบัสขนาด 8 บิต และเป็น ไอโอพอร์ทใช้งานทั่วไปเมื่อจะใช้งานเป็นอินพุตพอร์ทต้องส่งลอจิก “1” มาที่พอร์ทนี้ก่อนเพื่อบังคับให้ FET อยู่ในสถานะ off ดังรูปที่ 2-13



รูปที่ 2.8 แสดงโครงสร้าง port 2 (bit)

- พอร์ต 3 (ขา 10-17) มีทั้งหมด 8 บิต คือ ขา P3.7-P3.0 มีโครงสร้างคล้าย พอร์ต 1 ทำงานได้ 2 หน้าที่ คือ เป็นไอโอพอร์ท ถ้าจะโปรแกรมให้เป็น อินพุตพอร์ทต้องส่งลอจิก “1” มาที่พอร์ทนี้ก่อน



รูปที่ 2.9 แสดงโครงสร้าง port 3(bit)

อีกหน้าที่หนึ่งก็คือ ใช้ส่งสัญญาณควบคุมออกมาและรับสัญญาณเข้าไปสัญญาณต่าง ๆ ดังนี้

- **P3.0/RXD (Serial Input Port)** เป็นขาที่ใช้รับข้อมูลแบบอนุกรม (UART)
- **P3.1/TXD (Serial Output Port)** เป็นขาที่ใช้ส่งข้อมูลแบบอนุกรม (UART)
- **P3.2/INT0 (External Interrupt 0)** ใช้รับสัญญาณการขัดจังหวะจากภายนอกเบอร์ 0
- **P3.3/INT1 (External Interrupt 1)** ใช้รับสัญญาณการขัดจังหวะจากภายนอกเบอร์ 1
- **P3.4/T0 (Counter 0 External Input)** ขารับสัญญาณพัลส์อินพุตเข้าไปยังวงจรเคาท์เตอร์ 0
- **P3.5/T1 (Counter 1 External Input)** ขารับสัญญาณพัลส์อินพุตเข้าไปยังวงจรเคาท์เตอร์ 1
- **P3.6/WR (External Data Memory Write Strobe)** ขาสัญญาณควบคุมการเขียนข้อมูลลงหน่วยความจำข้อมูลภายนอก
- **P3.7/RD (External Data Memory Read Strobe)** ขาสัญญาณควบคุมการอ่านข้อมูล จากหน่วยความจำข้อมูลภายนอก
- **ALE (ขา 30)** เป็นขาส่ง Strobe สำหรับใช้ในการแล็ชแอดเดรสไบต์ ต่ำ (A7-A0) ที่ส่งออกมาจาก port 0 สัญญาณนี้จะ active ทุก ๆ 2 ครั้งในแมชชีนไซเกิล
- **PSEN (ขา 29)** เป็นขา Strobe สำหรับใช้อ่านข้อมูลจากโปรแกรมเมมโมรี่ภายนอก สัญญาณนี้จะส่งออกมา 2 ครั้งในแต่ละแมชชีนไซเกิลแต่ถ้าเป็นการอ่านโปรแกรมเมมโมรี่ภายในจะไม่มีสัญญาณออกที่ขา
- **EA (ขา 31)** ใช้เลือกหน่วยความจำโปรแกรมภายนอก ป้อน “ 0 “ จะอ่านโปรแกรมจากภายนอกชิพ ป้อน “1”จะอ่านโปรแกรมจากภายในชิพ
- **RST (ขา 9)** ขาริเซ็ตจะรีเซ็ตได้ก็ต่อเมื่อป้อนลอจิก “ 1 “ เข้าที่ขานี้ นานอย่างน้อย 2 แมชชีนไซเกิล

- XTAL1 (ขา 19) ใช้ต่อคริสตัลภายนอกโดยเป็นอินพุตเข้าสู่วงจรออสซิลเลเตอร์ภายใน
- XTAL2 (ขา 18) ใช้ต่อคริสตัลภายนอกโดยเป็นเอาต์พุตของวงจรออสซิลเลเตอร์ภายใน

2.5.4 พอร์ตสื่อสารข้อมูลแบบอนุกรมใน MCS-51

การใช้งานสื่อสารข้อมูลอนุกรมใน MCS-51 มีความสะดวกและคล่องตัวสูงทั้งนี้เนื่องจากผู้ใช้สามารถกำหนดการทำงานที่แตกต่างกันได้ถึง 4 ประเภทโดยสามารถกำหนดได้จากค่าของบิตในรีจิสเตอร์ใช้งานเฉพาะ SCON การใช้งานที่แตกต่างกัน 4 ประเภทนี้มีจุดประสงค์เพื่อความคล่องตัวในการรับหรือส่งข้อมูลแบบอนุกรมแต่ละประเภทดังนี้

- **โหมด 0** การทำงานของพอร์ตสื่อสารข้อมูลแบบอนุกรมในโหมด 0 ขา RXD จะสำหรับรับข้อมูลและส่งข้อมูล ส่วนขา TXD มีไว้เพื่อใช้สร้างสัญญาณ shift clock เพื่อกำหนดจังหวะในการรับและส่งข้อมูลในโหมดนี้ การรับส่งข้อมูลจะเป็นแบบ 8 บิต โดยเริ่มนับและส่งบิตต่ำสุดก่อน อัตราการรับส่งข้อมูลในการทำงานโหมด 0 ถูกกำหนดไว้ที่ $1/12$ ของความถี่ออสซิลเลเตอร์ที่ใช้ การทำงานของพอร์ตสื่อสารอนุกรมในโหมด 0 จะไม่มีบิตเริ่มต้นของข้อมูลและบิตสิ้นสุดของข้อมูล เพราะจังหวะในการรับและส่งข้อมูลถูกกำหนดจากสัญญาณ shift clock

- **โหมด 1** ทำงานแบบที่สองหรือการทำงานโหมด 1 นี้ มีการรับส่งข้อมูลครั้งละ 10 บิต ข้อมูลจะถูกส่งออกไปภายนอกผ่านทางขา TXD และ การรับส่งข้อมูลเข้ามาทางขา RXD ข้อมูลทั้ง 10 บิต ประกอบไปด้วยบิตเริ่มต้นของข้อมูล 1 บิต (มีค่าเป็น 0 เสมอ) บิตข้อมูล 8 บิต (รับและส่งบิตต่ำสุดก่อน) และบิตสิ้นสุดข้อมูลอีก 1 บิต (มีค่าเป็น 1 เสมอ) ในขณะที่ทำการรับส่งข้อมูล ค่าในบิตสิ้นสุดของข้อมูลที่ได้รับจะได้อ่านอยู่ในบิต RB8 ของ รีจิสเตอร์ใช้งานเฉพาะ SCON อัตราเร็วในการรับหรือส่งข้อมูลของพอร์ตสื่อสารข้อมูลแบบอนุกรมในโหมดนี้สามารถเปลี่ยนแปลงได้

- **โหมด 2** การทำงานแบบที่สามหรือโหมด 2 จะมีการรับและส่งข้อมูลครั้งละ 11 บิต ข้อมูลจะถูกส่งออกไปภายนอกผ่านทางขา TXD และรับเข้ามาผ่านทางขา RXD ข้อมูลที่รับและส่งทั้ง 11 บิตจะประกอบไปด้วยบิตเริ่มต้นของข้อมูล 1 บิต บิตข้อมูล 8 บิต ตามด้วยบิตที่ 9 ซึ่งเป็นบิตที่สามารถกำหนดให้มีค่าเป็นศูนย์หรือหนึ่งก็ได้ และบิตสุดท้ายคือบิตสิ้นสุดข้อมูล ดังนั้นจึงมีจำนวนบิตที่รับส่งทั้งหมด 11 บิต

- **โหมด 3** การทำงานของพอร์ตสื่อสารข้อมูลแบบอนุกรมแบบสุดท้าย คือการทำงานในโหมด 3 ในการทำงานโหมดนี้ข้อมูลจำนวน 11 บิตจะถูกส่งออกทางขา TXD และรับข้อมูลผ่านทางขา RXD ข้อมูลทั้ง 11 บิตประกอบไปด้วยบิตเริ่มต้นของข้อมูล 1 บิต บิตข้อมูล 8 บิตตามด้วยบิตที่ 9 ซึ่งเป็นบิตที่สามารถกำหนดค่าได้เหมือนในโหมดที่ 2 และบิตสุดท้ายคือบิตสิ้นสุดข้อมูล อัตราเร็วในการรับหรือส่งข้อมูลสามารถเปลี่ยนแปลงได้ ซึ่งในโหมดนี้สามารถกำหนดอัตราเร็วในการรับหรือส่งข้อมูลได้ตามความต้องการของผู้ใช้งาน

อัตราเร็วในการรับส่งข้อมูล

Baud rate หมายความว่าถึงอัตราเร็วในการรับหรือส่งข้อมูล โดยใน MCS-51 ค่าอัตราเร็วในการรับส่งข้อมูลจะมีค่าเท่าใดขึ้นอยู่กับการทำงานในแต่ละโหมดของ port สื่อสารข้อมูลแบบอนุกรมดังนี้

$$\text{Baud rate โหมด 0} = \frac{\text{ความถี่ออสซิลเลเตอร์ที่ใช้}}{12}$$

หากใช้คริสตอลความถี่ 12 MHz ค่า baud rate ของ port สื่อสารข้อมูลแบบอนุกรมโหมด 0 จะมีค่าสูงถึง 1 MHz. ในโหมดที่ 2 ค่า baud rate ขึ้นอยู่กับค่าของ บิต SMOD ที่อยู่ในรีจิสเตอร์ใช้งานเฉพาะ PCON โดยบิต SMOD = 0 ค่า baud rate จะเป็น 1/64 ของความถี่ออสซิลเลเตอร์ที่ใช้ บิต SMOD = 1 ค่า baud rate จะเป็น 1/32 ของความถี่ออสซิลเลเตอร์ที่ใช้หลังจากการ reset MCS-51 ค่าในบิต SMOD จะเป็น 0 เสมอและเราสามารถเขียนสูตรสำหรับคำนวณค่า baud rate ได้ดังสมการนี้

$$\text{Baud rate โหมด 2} = \frac{[2 (\text{SMOD}) \times (\text{ความถี่ออสซิลเลเตอร์ที่ใช้})]}{64}$$

หากใช้คริสตอลความถี่ 12 MHz baud rate สูงสุดในการทำงานโหมดนี้คือ 375 K

อธิบายการทำงานพอร์ตสื่อสารข้อมูลแบบอนุกรม (โหมด 1 เพิ่มเติม)

การทำงานของพอร์ตสื่อสารข้อมูลแบบอนุกรมโหมด 1 ข้อมูล 10 บิต ถูกส่งออกไปภายนอกผ่านขา TXD และรับเข้ามาจากภายนอกผ่านทางขา RXD ข้อมูลที่รับหรือส่งจะประกอบไปด้วย บิตเริ่มต้นของข้อมูล 1 บิต (0) บิตข้อมูล 8 บิตที่เริ่มรับหรือส่งด้วยบิตต่ำสุดก่อน (LSB first) ตามด้วยบิตสิ้นสุดของข้อมูลอีก 1 บิต (1)

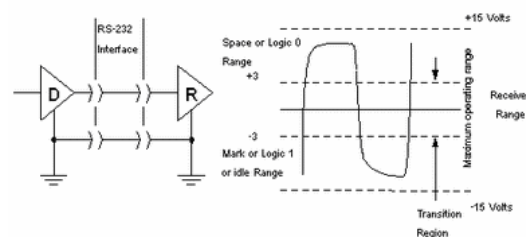
ในการรับข้อมูล บิตสิ้นสุดของข้อมูลจะถูกนำไปไว้ในบิต RB 8 ของรีจิสเตอร์ใช้งานเฉพาะ SCON ค่า baud rate ถูกกำหนดโดยอัตราการเกิด overflow ของไทม์เมอร์ 1 (timer 1 overflow rate) ส่วนใน 8052 ซึ่งมีไทม์เมอร์ 2 อย่างใดอย่างหนึ่งเพิ่มขึ้นมา ค่า baud rate สามารถถูกกำหนดโดยอัตราการเกิด overflow ของไทม์เมอร์ 1 หรือ ไทม์เมอร์ 2 อย่างใดอย่างหนึ่งหรือสองอย่าง (ใช้ตัวหนึ่งสำหรับการกำหนด + ในการส่งข้อมูลและอีกตัวหนึ่งสำหรับการรับข้อมูล) โดยใช้ บิต RCLK และบิต TCLK เป็นตัวเลือก

2.6 การสื่อสาร

เนื่องจาก RS-422 และ RS-485 เป็นการส่งข้อมูลในระบบสมดุล (Balanced System) เหมือนกัน จึงนำมากล่าวไว้ในบทเดียวกัน และต่อไปนี้จะแทนมาตรฐาน EIA/TIA - 422 ด้วย RS-422 และแทนมาตรฐาน EIA/TIA - 485 ด้วย RS-485

2.6.1 ตัวส่งแบบไม่สมดุล (Unbalanced line driver)

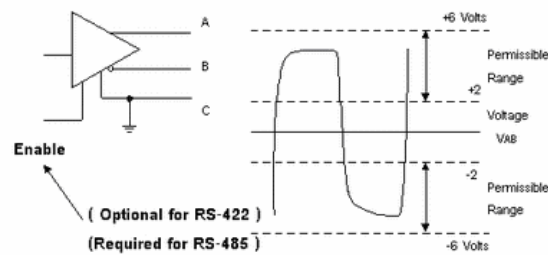
ระดับสัญญาณที่ส่งในระบบไม่สมดุล RS-232 เป็นระบบที่วัดระดับแรงดันสัญญาณเทียบกับสายกราวด์ ยกตัวอย่างเช่น ในการส่งข้อมูล (Transmission Data : TD) จากอุปกรณ์ (Data Terminal Equipment : DTE) ผ่านหัวต่อแบบ DB-25 จะส่งสัญญาณข้อมูลทางขา 2 โดยการวัดระดับแรงดันของสัญญาณจะวัดเทียบกับสายกราวด์ (Signal ground) ที่ขา 7 ถ้าสายส่งข้อมูลอยู่ในสถานะ idle ระดับแรงดันจะมีค่าเป็นลบ และเมื่อทำการส่งข้อมูลระดับแรงดันจะเปลี่ยนแปลงในช่วงค่าบวกและค่าลบ โดยมีระดับแรงดันอยู่ในช่วง ± 5 V ถึง ± 15 V รูปที่ 2.10 แสดงการทำงานของตัวรับของ RS-232 โดยจะทำงานในระดับแรงดันช่วง +3 ถึง +12 และ -3 ถึง -12



รูปที่ 2.10 RS-232 Interface Circuit

2.6.2 ตัวส่งแบบสมดุล (Balanced driver)

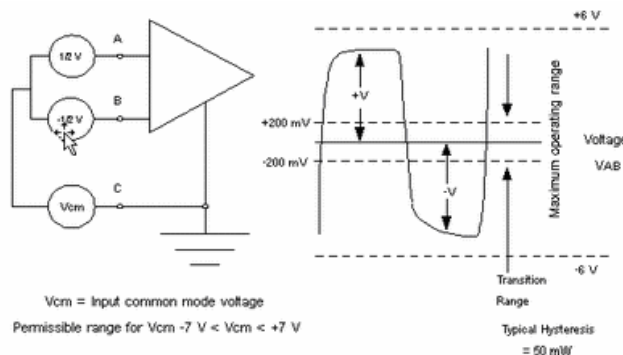
ระบบสมดุลจะส่งสัญญาณผ่านสาย 2 เส้น รูปที่ 2.11 แสดงรูปแบบและระดับแรงดัน ตัวส่งจะส่งแรงดันช่วง 2-6 V ที่เอาต์พุตระหว่าง A และ B ระบบจะมีการเชื่อมต่อสายกราวด์ 1 เส้น สายกราวด์ในระบบนี้ไม่ได้ใช้ในการส่งสัญญาณหรือหาสถานะลอจิกของข้อมูล แต่สายกราวด์มีความสำคัญคือใช้เป็นจุดอ้างอิงของระบบ การวัดสัญญาณข้อมูลจะถูกวัดเทียบกับสายกราวด์ ในการส่งข้อมูลตัวส่งจะได้รับสัญญาณหนึ่งเรียกว่าสัญญาณควบคุม หรือ Enable signal ซึ่งเชื่อมต่อกันระหว่างตัวส่งและเทอร์มินอล A และ B ที่เอาต์พุต ตัวส่งจะส่งสัญญาณควบคุมซึ่งเปรียบเสมือนเป็นเอาต์พุตที่ 3 นอกจากการส่งสถานะลอจิก 1 หรือ 0 ให้กับเทอร์มินอล ถ้าสัญญาณควบคุมอยู่ในสถานะ OFF จะหมายถึงตัวส่งตัวนั้นไม่ได้ต่อกับสายสัญญาณและไม่สามารถส่งสัญญาณข้อมูลได้ หรืออยู่ในสถานะ disable หรือ tri-state



รูปที่ 2.11 Balanced Differential Output Line Driver

2.6.3 ตัวรับแบบสมดุล (Balanced line receiver)

ตัวรับในระบบสมดุลจะถูกเชื่อมต่อด้วยสายส่งสัญญาณข้อมูลและสายกราวด์ โดยจะรับสถานะของสัญญาณได้โดยวัดความแตกต่างของระดับแรงดันที่สายอินพุทของตัวรับ หรือ วัดระหว่างเทอร์มินอล A และ B (V_{ab}) ดังแสดงในรูปที่ 2.12 ถ้าแรงดันมีขนาดมากกว่า +200 mV จะถูกกำหนดให้มีสถานะเป็นลอจิกค่าหนึ่ง และถ้ามีขนาดน้อยกว่า -200 mV จะถูกกำหนดให้มีสถานะตรงข้าม เนื่องจากอาจเกิดการลดทอนสัญญาณ (Attenuate) ขึ้นในสายส่งที่ตัวรับจึงถูกออกแบบให้สามารถรับความแตกต่างระดับแรงดันของสัญญาณได้ในช่วงที่กว้างกว่าตัวส่งคือ +200 mV ถึง +6 V ในขณะที่แรงดันที่ตัวส่งอยู่ในช่วง +2 V ถึง +6 V

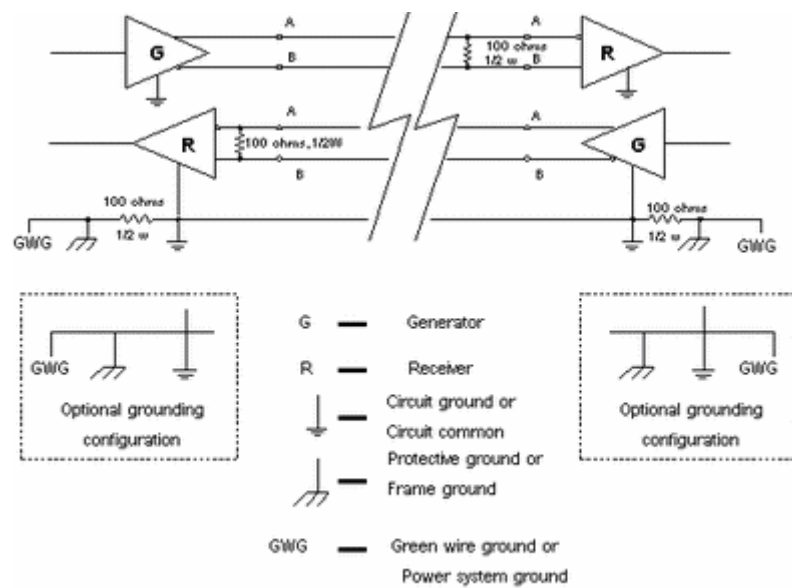


รูปที่ 2.12 Balanced Differential Input Line Receiver

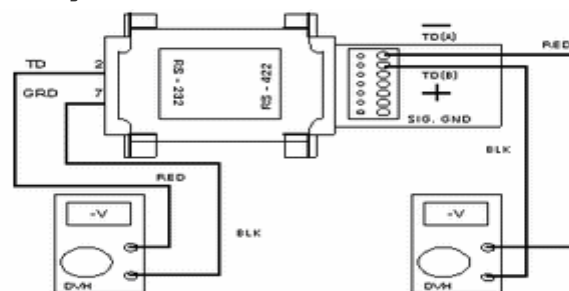
2.6.4 การส่งข้อมูลตามมาตรฐาน RS-422(EIA Standard RS-422 Data Transmission)

รูปที่ 2.13 เป็นการเชื่อมต่อทางมาตรฐาน RS-422 แบบ 4 สาย ซึ่งประกอบด้วยสายตัวนำ 5 เส้น (สายสัญญาณ 4 เส้น และสายกราวด์ 1 เส้น) โดยสถานะในการส่งดังนี้

- ถ้าแรงดัน A เทียบ B (V_{ab}) มีค่าเป็นลบ จะถูกกำหนดให้มีสถานะลอจิก 1 หรือ MARK หรือ OFF
- ถ้าแรงดัน A เทียบ B (V_{ab}) มีค่าเป็นบวก จะถูกกำหนดให้มีสถานะลอจิก 0 หรือ SPACE หรือ ON



รูปที่ 2.13 Typical RS-422 Four-Wire Network

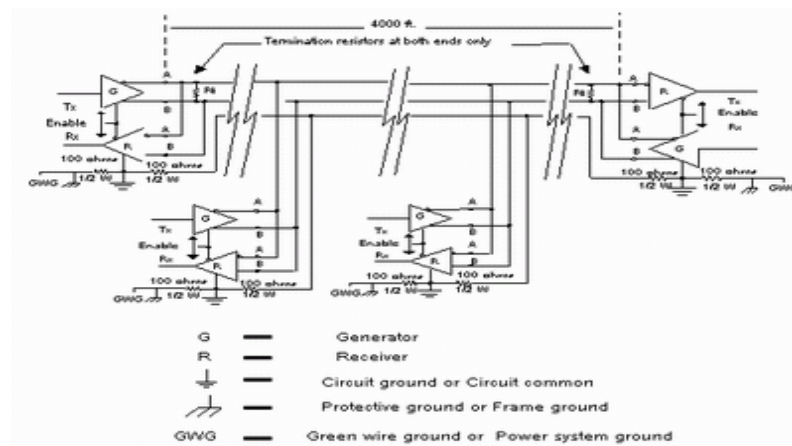


Note: Under "idle" conditions it is possible to determine which terminal is "A" and which is "B"

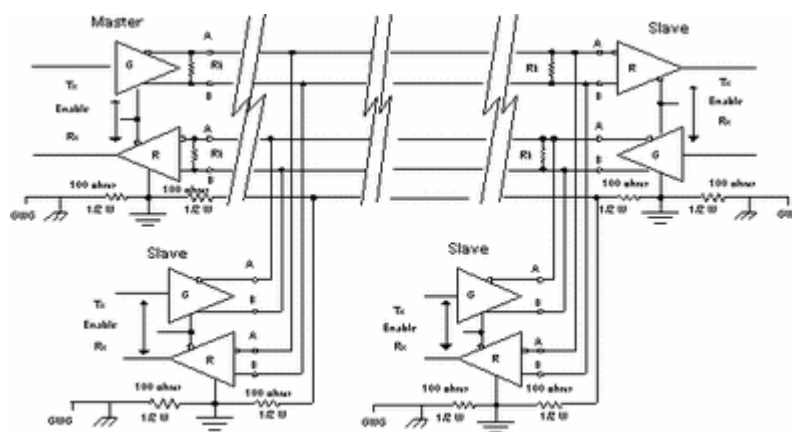
รูปที่ 2.14 แสดงระดับแรงดันของวงจรสมมูลของตัวเปลี่ยนจาก RS-232 เป็น RS-485

2.6.5 การส่งข้อมูลตามมาตรฐาน RS-485(EIA Standard RS-485 Data Transmission)

RS-485 เป็นการส่งข้อมูลในระบบสมมูล สายสัญญาณ 1 คู่สายสามารถติดต่ออุปกรณ์ได้ถึง 32 ตัว คุณสมบัติของอุปกรณ์ในระบบ RS-422 และ RS-485 มีลักษณะคล้ายคลึงกัน สำหรับในระบบ RS-485 สามารถทนแรงดันระหว่างสายสัญญาณและสายกราวด์ หรือ Common Mode Voltage หรือ V_{cm} ได้ในช่วง - 7 V ถึง +12 V ซึ่งมากกว่าอุปกรณ์ในระบบ RS-422



รูปที่ 2.15 Typical RS - 485 Two Wire Multidrop Network



รูปที่ 2.16 แสดงระบบ RS-485 แบบ 4 สาย

2.6.6 การควบคุมการส่งข้อมูลของอุปกรณ์ RS-485 (Send Data Control of a RS-485 Device)

ผลิตภัณฑ์ตัวเปลี่ยน RS-232 เป็น RS-485 (RS-232 to RS-485 Converter) และการ์ดอนุกรม RS-485 (RS-485 serial cards) มีวงจรพิเศษในการใช้สัญญาณข้อมูลเป็นตัว enable ให้กับตัวส่ง (RS-485 driver) รูปที่ 2.17 แสดง Timing diagram ของสัญญาณที่ใช้ควบคุมตัวเปลี่ยน (Converter) จาก diagram แสดงให้เห็นว่ามีช่วงเวลาหนึ่งที่เวลาหนึ่งหลังการส่งข้อมูลบิตสุดท้ายและก่อนการ disable ของตัวส่ง ถ้าระยะเวลานั้นสั้นเกินไปหรือตัวส่งอาจถูก disable ก่อนที่ตัวส่งจะส่งข้อมูลเสร็จ อาจทำให้ข้อมูลในส่วนท้ายสูญหายได้ และถ้ามีขนาดยาวเกินไปจะทำให้ระบบเปลี่ยนสายสัญญาณจากส่งเป็นรับก่อนที่โหนดพร้อมจะรับข้อมูลหรือตัวรับเชื่อมต่อเข้ากับสายส่งและรับข้อมูลซ้ำไม่ทันต่อสัญญาณที่ส่งมาถึง ทำให้ไม่ได้รับข้อมูลในส่วนแรก ดังนั้นควรกำหนดช่วงเวลาดังกล่าวให้เหมาะสม โดยส่วนมากมักเท่ากับความยาวหนึ่งตัวอักษรที่อัตราการส่งข้อมูลนั้นๆ

